

Westmoor University Research Data Lakehouse and Retrieval Platform

Detailed Design

Architecture, requirements, and validation criteria for build

Version	1.1
Status	Design phase — build not started
Last updated	2026-10-01
Prepared by	Research Computing and Data Services (RCDS), Platform Architecture Group
Companion documents	RDLR Conceptual Design v1.2; Westmoor Research Data Classification Standard (RDSCS-2025)

Changes since version 1.0

- Section 2.2: NFR-6 references updated; NFR-9 total run-rate envelope revised.
- Sections 7.3 and 13: consent-withdrawal handling of snapshots, tags and versions.
- Sections 8.1 and 18: catalog deployment topology.
- Section 11: notebook credential model.
- Sections 15 and 17.1: vector index sizing and cost lines.
- Section 22.2: NFR-10 acceptance criterion.

Table of Contents

1. Purpose and Scope
2. Requirements
3. Foundational Principles
4. Data Classification Tiers
5. Target Architecture
6. Identity, Projects and Roles
7. Storage and Table Layout
8. Catalog and Metadata
9. Ingestion — Write-Audit-Publish
10. Access Control and Policy Enforcement
11. Analysis Environments
12. Embargo and Data-Sharing Agreements

- 13.Consent Withdrawal
 - 14.Scholar Assist — Retrieval-Augmented Generation
 - 15.Vector Index Sizing
 - 16.Audit Plane
 - 17.Cost Model and Chargeback
 - 18.Resilience and Disaster Recovery
 - 19.Prior Art and Reference Architecture
 - 20.Confirmed Decisions
 - 21.Pending Backlog
 - 22.Validation and Acceptance Criteria
 - 23.Readiness Assessment
 - 24.Build Phases
-

1. Purpose and Scope

This document describes the architecture of the Westmoor Research Data Lakehouse and Retrieval Platform (RDLR): a governed, multi-tenant platform that consolidates research datasets, publications and electronic lab notebooks from all eleven faculties into a single lakehouse, and exposes a retrieval-augmented generation (RAG) service, **Scholar Assist**, over the text-bearing portion of that estate. It is written to implementation level for the storage, ingestion, access-control and audit layers, and to architectural level for the RAG and chargeback components. Section 23 states plainly where the design is and is not yet ready for build.

RDLR is not a departmental data store. It is an institution-level platform that provides:

- A lakehouse on Amazon S3 with Apache Iceberg tables and an Iceberg REST catalog, replacing roughly forty faculty-run file shares and departmental databases.
- Project-scoped access control: every dataset belongs to exactly one research project, and access follows project membership, never faculty or department affiliation.
- Embargo enforcement for unpublished results, patent-pending work and sponsor-restricted outputs.
- Data-sharing agreement (DSA) enforcement across the agreement lifecycle: onboarding, permitted users and purposes, term, and end-of-term destruction or return of data.
- Scholar Assist: question answering with citations over the publications, lab notebooks and dataset documentation that the requesting researcher is entitled to see.
- Monthly cost chargeback to projects and grant accounts.
- An audit plane that records every data access, grant change and RAG retrieval, physically and administratively separated from the data plane it observes.

In scope

Approximately 3,000 active researchers (faculty, postdoctoral fellows, research staff and doctoral candidates) across 11 faculties; approximately 2 PB of research data at launch (about 1.1 billion objects); approximately 640 active research projects; inbound and outbound DSAs with approximately 120 partner institutions and sponsors.

Out of scope

- Teaching and learning data, student records and the learning management system.
- Clinical systems of record at Westmoor Medical Center. De-identified extracts arrive through the standard

ingestion path; identified PHI does not enter the platform.

- Long-term preservation and public deposit, which remain the responsibility of the Institutional Repository. RDLR feeds the repository but does not replace it.
- HPC scratch file systems. HPC jobs read from and publish to the lakehouse, but scratch storage is not governed by RDLR.

2. Requirements

Requirements are grouped into Functional Requirements (what the platform must do) and Non-Functional Requirements (the quality attributes it must exhibit). Each requirement carries an ID that is used again in Section 22 (Validation and Acceptance Criteria) and Section 23 (Readiness Assessment).

2.1 Functional Requirements

ID	Requirement
FR-1	The platform shall ingest research datasets (tabular files, instrument outputs, imaging collections, code and documentation) from registered sources into Iceberg tables or managed file collections, through registered pipelines only.
FR-2	Every dataset shall be registered to exactly one project, with a named PI owner, a classification tier (Section 4), and optional embargo and DSA linkage, before any of its data is published.
FR-3	Newly ingested data shall not be visible on a table's main branch until schema, classification and integrity validation have passed (write-audit-publish).
FR-4	A user shall be able to read a dataset only if they hold a role (PI, Member, Analyst, Viewer) on the owning project, or are named on an active DSA that grants access to it. Faculty or department affiliation alone shall never grant access.
FR-5	While a dataset or publication is under embargo, neither its content nor its descriptive metadata (title, abstract, keywords, PI name) shall be visible to anyone outside the owning project until the embargo lift date.
FR-6	A dataset derived from one or more embargoed datasets shall inherit the embargo of its source dataset(s).
FR-7	Inbound DSAs shall be registered in the Project Registry with permitted users, permitted purposes, term and governing obligations; access to DSA-governed datasets shall be limited to the users named on the DSA.
FR-8	Researchers shall be able to ask natural-language questions over publications, lab notebook entries and dataset documentation through Scholar Assist, and receive answers with citations that resolve to the exact source version used.
FR-9	Scholar Assist shall restrict retrieval, before similarity search executes, to content the requesting user is entitled to under FR-4, FR-5 and FR-7.

ID	Requirement
FR-10	The platform shall provide a Discovery Portal through which researchers can find datasets and request access from the owning PI.
FR-11	Any analysis cited in a publication shall be re-runnable against the exact table snapshots it used for at least five years after publication.
FR-12	A PI shall be able to register the withdrawal of a human-subject participant's consent, identified by the participant's project pseudonym, and the platform shall execute erasure in accordance with NFR-7.
FR-13	The platform shall produce a monthly chargeback statement per project, mapped to the project's cost centre or grant account.
FR-14	Every data read (SQL query, file read, RAG retrieval), grant change, embargo change, DSA event and consent-withdrawal event shall be recorded in the audit plane with principal, object, action, outcome and timestamp.
FR-15	Researchers shall have a managed notebook environment (JupyterHub) with Spark and Trino access to the datasets they are entitled to.

2.2 Non-Functional Requirements

ID	Requirement
NFR-1	The platform shall support 3,000 researchers, at least 600 concurrent interactive sessions at peak, and approximately 2 PB / 1.1 billion objects at launch, growing about 35% per year, without re-architecture for three years.
NFR-2	Interactive SQL, the notebook environment and Scholar Assist shall each be available 99.9% per calendar month, excluding announced maintenance windows of no more than 4 hours per month.
NFR-3	Scholar Assist p95 time-to-first-token shall be ≤ 2.5 s, and the p95 retrieval stage (entitlement resolution, search and rerank) ≤ 400 ms, at a sustained 20 queries per second.
NFR-4	Batch ingestion shall be published within 24 h of landing; instrument streams within 60 min; new or changed text sources shall be searchable in Scholar Assist within 6 h of publication.
NFR-5	Content classified Restricted or Controlled shall not be transmitted to, or processed by, any service outside Westmoor-controlled AWS accounts.
NFR-6	Datasets tagged CUI shall be encrypted at rest using FIPS-validated cryptography (NIST SP 800-171 Rev. 2 requirements 3.13.11 and 3.13.16), with customer-managed AWS KMS keys rotated every 90 days in accordance with the Westmoor Cryptographic Key Management Standard.
NFR-7	When a participant's consent withdrawal is registered, all research data records relating to that participant (rows, files and derived records) shall be erased from every platform data store within 30 days, as required by the approved IRB protocols and, for EU-resident participants, GDPR Article 17. Audit metadata is governed by NFR-8.

ID	Requirement
NFR-8	Audit records shall be immutable, tamper-evident and retained for at least seven years.
NFR-9	Primary storage cost shall not exceed USD 10,000 per month at 2 PB; total platform run-rate shall not exceed USD 80,000 per month at launch scale.
NFR-10	At least 95% of monthly platform cost shall be attributed to individual projects; the unattributed shared remainder shall not exceed 5%.
NFR-11	Region-loss RPO ≤ 24 h and RTO ≤ 24 h for all tiers; accidental deletion or corruption of a table shall be recoverable within 4 h.
NFR-12	A new faculty source shall be onboarded by registering a source and a pipeline configuration, without code changes to the ingestion framework, policy layer or RAG service.

3. Foundational Principles

These ten principles govern every design decision in the platform, from bucket layout to the shape of a Scholar Assist citation. Where a later section appears to conflict with one of these, the principle wins.

#	Principle
P1	The project is the unit of access. Data belongs to a project; people reach data through project roles or DSAs, never through organisational position.
P2	Policy before data. Entitlement gates the query or the similarity search itself, not the results after retrieval.
P3	One copy, many engines. Iceberg tables are read in place by Trino, Spark and the RAG indexer; no engine keeps a private copy.
P4	Publish only what has been audited. Nothing reaches a main branch without passing validation.
P5	Classification travels with the data. The tier determines the bucket, the key, and which services may process the data.
P6	Embargo is confidentiality, not just access control. The existence and description of embargoed work are themselves protected.
P7	The audit plane sits outside the data plane. Separate AWS account, separate operators, write-once storage.
P8	Every answer is traceable. A Scholar Assist citation resolves to a chunk, a source object and a source version.
P9	Costs follow projects. Every metered resource carries a project attribution.

#	Principle
P10	Configuration over code. Sources, pipelines, data contracts and policies are declarative configuration.

4. Data Classification Tiers

Every dataset carries exactly one tier from the Westmoor Research Data Classification Standard. The tier is chosen by the PI at registration, confirmed by the classification scan during ingestion (Section 9), and determines storage location, encryption key, permitted engines and Scholar Assist eligibility.

Tier	Typical content	Bucket / account	Encryption key	Permitted engines	Scholar Assist	Processing outside Westmoor accounts
Public	Published papers, open datasets, public code	rdlr-public, data account	AWS-managed KMS key	All	Indexed	Permitted
Internal	Unpublished drafts without sponsor restriction, lab notebooks (default), internal technical reports	rdlr-internal, data account	Customer-managed key (CMK) per faculty	All	Indexed	Permitted under enterprise contract terms
Restricted	De-identified human-subject data, sponsor-confidential results, DSA-governed inbound data	rdlr-restricted, data account	CMK per project	Trino, Spark, notebooks	Indexed (documentation and notebook text only; tabular rows never embedded)	Not permitted (NFR-5)
Controlled	CUI, export-controlled (EAR/ITAR) data, HIPAA limited data sets under a data use agreement	rdlr-controlled, separate Controlled-enclave account	CMK per project, FIPS endpoints	Trino and Spark inside the Controlled enclave only	Never indexed	Not permitted (NFR-5)

Tier rules.

- The classification scanner may raise a tier automatically; lowering a tier requires Research Data Steward

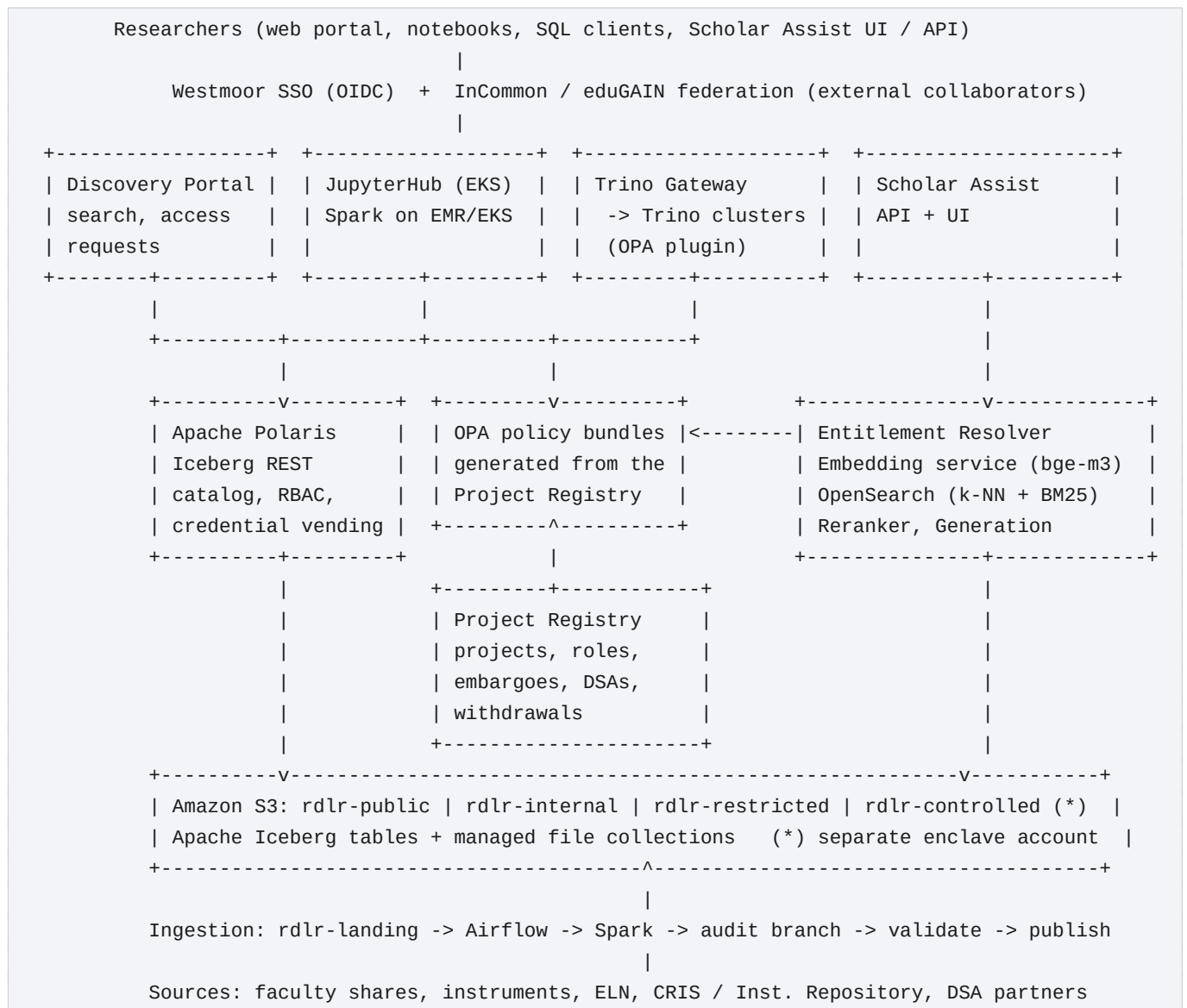
approval and is audit-logged.

- A derived dataset takes the most restrictive tier among its inputs. This is applied at publish time from lineage (Section 12.1) and cannot be overridden by the PI.
- Moving a dataset between tiers is a copy into the target bucket followed by a catalog re-pointing commit and deletion from the source bucket; it is never an in-place relabel, because the bucket and key are the enforcement boundary.

Rationale. The Controlled tier is placed in its own AWS account so that the NIST SP 800-171 assessment boundary is the enclave account and nothing else. Excluding Controlled content from Scholar Assist keeps the RAG service, the embedding fleet and the vector index entirely outside that boundary. Restricted tabular rows are never embedded because row-level policies (Section 10) cannot be expressed against a vector index; documentation about a Restricted dataset is embedded, and is protected by the project-level pre-filter in Section 14.3.

5. Target Architecture

Every data operation, from every engine, for every researcher, passes through the catalog and the policy layer. No engine or user reaches an S3 data bucket with standing credentials.



Audit plane (separate AWS account): CloudTrail S3 data events, Trino event listener, Polaris events, OPA decision logs, Scholar Assist retrieval logs, Registry events
-> Firehose -> S3 Object Lock

Figure 1 — Target architecture. Every arrow is an authenticated, audited hand-off.

Layer responsibilities

- **Access surfaces** — Discovery Portal, JupyterHub, SQL clients via the Trino Gateway, and the Scholar Assist UI and API. None holds standing data credentials.
- **Identity** — Westmoor SSO (OIDC) for staff and students; InCommon / eduGAIN federation for external collaborators named on DSAs.
- **Project Registry** — system of record for projects, roles, embargoes, DSAs and consent withdrawals. Source of all grants and policy bundles.
- **Catalog** — Apache Polaris (Iceberg REST). Namespace-level RBAC, table metadata, and short-lived, table-scoped credential vending for S3 reads and writes.
- **Policy** — Open Policy Agent (OPA) bundles generated from the Registry; row filters and column masks applied by the query engines.
- **Compute** — Trino (interactive SQL, two clusters behind Trino Gateway), Spark on EMR on EKS (batch and ingestion), JupyterHub (notebooks).
- **Scholar Assist** — Entitlement Resolver, embedding service, OpenSearch hybrid index, reranker, generation, citation service.
- **Storage** — S3 buckets per tier; Iceberg format v2 tables and Iceberg-manifested file collections.
- **Audit plane** — separate AWS account; write-once storage; queried by the Audit Office and Research Integrity Office.

6. Identity, Projects and Roles

6.1 Identity

Field	Value	Notes
wid	Westmoor ID, from SSO sub claim	Immutable primary key used everywhere in the platform. Never reassigned.
upn	e.g. j.okafor@westmoor.edu	Display and login only; may change on name change.
affiliation	faculty, department, appointment type	Informational; never used in access decisions (P1).
external	boolean	True for federated collaborators. External users may hold only the Analyst role.
orcid	optional	Used in citation display and access-request forms.

Authentication is OIDC via Westmoor SSO with MFA enforced. External collaborators federate through InCommon or eduGAIN; access to Restricted datasets additionally requires the REFEDS MFA assurance profile to be asserted by the home institution.

6.2 Project Registry

The Project Registry is a small PostgreSQL-backed service (Multi-AZ Amazon RDS) that holds projects, role assignments, embargo records, DSA records and consent-withdrawal records. Projects are created from the grants management system when an award is set up, or manually by a Research Data Steward for unfunded work. Each project carries a cost centre or grant account for chargeback (Section 17).

6.3 Roles

Role	Data read	Data write	Grant management	Notes
PI	All project datasets	Yes (via pipelines and project scratch)	Assigns roles, sets embargoes, registers withdrawals	Exactly one per project. Co-PIs are Members with delegated grant rights.
Member	All project datasets	Yes	No (unless delegated)	
Analyst	Datasets explicitly granted	Project scratch only	No	Used for DSA-named and external users.
Viewer	Dataset documentation only	No	No	For advisory committees and examiners.
Research Data Steward	Metadata only; data via break-glass	No	Approves tier lowering and DSA activation	One or more per faculty. Break-glass is time-boxed and audited.

6.4 Group provisioning

Role changes in the Registry are pushed to the IdP as groups named `rdlr-prj-<project_id>-<role>` via SCIM within five minutes. Polaris principal roles are bound to these groups, and the OPA bundle (Section 10.3) is regenerated on every Registry change. Removal of a role therefore takes effect in the catalog within five minutes and in the policy layer within one bundle cycle.

7. Storage and Table Layout

7.1 Buckets and prefixes

One bucket per tier, with a uniform prefix layout:

```
s3://rdlr-<tier>/<faculty>/<project_id>/<dataset_id>/data/...
s3://rdlr-<tier>/<faculty>/<project_id>/<dataset_id>/metadata/...
s3://rdlr-<tier>/<faculty>/<project_id>/_scratch/<wid>/...
```

Bucket policies deny all access except from the Polaris credential-vending role, the ingestion role and the break-glass role. S3 Block Public Access is enforced at the account level, including on `rdlr-public` (public datasets are served through the Institutional Repository, not directly from the lakehouse).

7.2 Iceberg conventions

Setting	Value	Reason
Format version	2	Row-level deletes; branches and tags.
File format	Parquet, ZSTD	Engine-neutral; good compression on instrument data.
Target file size	512 MB	Reduces object count and planning cost.
Row-level deletes	Merge-on-read for tables > 1 TB; copy-on-write otherwise	Keeps large-table deletes cheap.
Compaction	Nightly <code>rewrite_data_files</code> per table with > 50 small files	Controls small-file growth from streams.
Orphan cleanup	Weekly <code>remove_orphan_files</code> , 7-day safety window	Reclaims files from failed audit branches.

7.3 Snapshots, tags and versions

Iceberg snapshots are retained for 400 days, which covers a full academic year plus a rollback buffer for NFR-11. A snapshot referenced by a publication tag (`pub-<doi-suffix>`) is retained indefinitely, which satisfies FR-11 for any analysis whose tables were tagged at submission. S3 Versioning is enabled on all data buckets (it is required for cross-region replication, Section 18), with noncurrent object versions expired after 90 days.

For human-subject tables and file collections, these retention periods are subordinate to the consent-withdrawal procedure (Section 13): a withdrawal removes the participant's data from every snapshot, tag and object version, in both regions, regardless of the retention that would otherwise apply.

7.4 Managed file collections

Non-tabular data (imaging stacks, instrument raw output, video) is stored as objects under the dataset prefix and described by an Iceberg *manifest table* with one row per file: path, size, checksum, content type, acquisition metadata and, for human-subject data, the participant pseudonym. Reads go through the manifest table, so file collections inherit catalog access control, snapshots and tags exactly as tables do.

7.5 Object profile

From the inventory of the faculty file shares to be migrated: approximately 1.1 billion objects, median size 340 KB, with about 23% of objects smaller than 128 KB (Iceberg metadata, notebook attachments, small instrument outputs). Bytes are dominated by imaging and genomics collections, which hold about 70% of total volume in about

4% of objects.

8. Catalog and Metadata

8.1 Catalog service

The Iceberg REST catalog is Apache Polaris. Polaris is stateless apart from its metastore, and runs as three replicas spread across three Availability Zones on the shared EKS cluster in us-east-1, behind an internal Application Load Balancer. The metastore is an Amazon Aurora PostgreSQL cluster with one writer and two readers across three AZs; writer failover to a reader typically completes in under a minute.

To remove the regional dependency, a second Polaris deployment runs in us-west-2 with its own Aurora PostgreSQL cluster. The two metastores are kept in sync by AWS DMS ongoing replication in both directions, with last-writer-wins conflict resolution on the row update timestamp, so either region can accept catalog commits. Trino, Spark and the Entitlement Resolver use the catalog endpoint in their own region; the us-west-2 Spark batch cluster, added for overflow capacity during term-start ingestion peaks, commits through the us-west-2 catalog. Catalog failover is a DNS change; catalog RPO is the DMS replication lag (seconds).

The catalog is on the path of every data operation: Trino and Spark call it for table loads and commits; every ingestion publish is a catalog commit; every S3 read and write uses credentials vended by Polaris with a 15-minute lifetime; and the Scholar Assist Entitlement Resolver lists a user's readable namespaces from Polaris on a cache miss.

8.2 Catalog structure

catalog:	rdlr_public rdlr_internal rdlr_restricted	(Controlled enclave runs its own Polaris)
namespace:	<faculty>.<project_id>	
table:	<dataset_id>	(Iceberg table or file-collection manifest table)

Polaris principal roles map one-to-one to Registry project roles. Catalog roles grant `TABLE_READ_DATA` (Member, Analyst where granted) or `TABLE_WRITE_DATA` (ingestion and project scratch writers) at namespace or table level.

8.3 Discovery Portal

The Discovery Portal is built on OpenMetadata and indexes catalog and Registry metadata nightly. Every registered dataset, including embargoed and Restricted datasets, is listed with its title, PI, abstract, keywords, classification tier and embargo-lift date, so that prospective collaborators can find it and request access through the PI. Schemas, sample rows and file listings are shown only to users who already hold read access. Access requests create a Registry ticket routed to the PI; approval creates the role assignment and, for DSA-governed data, is blocked unless the requester is named on an active DSA.

9. Ingestion — Write-Audit-Publish

All ingestion follows a single write-audit-publish (WAP) pattern implemented with Iceberg branches. It is the only path by which data enters a main branch.

1. Land	Source pushes or is pulled into <code>s3://rdlr-landing/<source_id>/<run_id>/</code> with a source manifest (file list, sizes, SHA-256 checksums, row counts).
2. Write	Airflow triggers a Spark job (EMR on EKS) that writes the run into a branch <code>audit-<run_id></code> of the target table. Data files are written once; the branch is only a metadata reference.
3. Audit	Validation on the branch: a. Schema contract: column names, types, nullability, declared keys. b. Integrity: row counts and checksums against the source manifest. c. Classification scan: Amazon Macie custom identifiers plus an NER detector for direct identifiers (names, MRNs, SSNs, addresses). A direct identifier in a dataset declared de-identified fails the run. d. Referential checks declared in the data contract. e. Tier check: scanner result must be <code><=</code> declared tier, else tier is raised and the run is held for steward review.
4. Publish	On pass: fast-forward main to the branch head in one atomic catalog commit. On fail: branch retained 14 days for diagnosis; PI and source owner notified; nothing becomes visible on main. Orphaned files are reclaimed by the weekly orphan cleanup (Section 7.2).

Idempotency and replay. Each run is keyed by `run_id`; re-running a run writes a new branch and the earlier branch is discarded. Publishing is a compare-and-swap on the table's main reference, so a concurrent publish fails cleanly and is retried after re-validation against the new base.

Streams. Instrument streams are written as micro-batches every 5 minutes into a rolling audit branch, validated and published every 30 minutes. Worst-case latency from landing to publish is therefore about 40 minutes, within NFR-4.

Onboarding. A new source is a YAML source definition (connection, schedule, landing prefix) plus a data contract (schema, keys, tier, participant-key column if any). Neither requires code changes to the framework (NFR-12).

Why this pattern. Branches share data files with main, so validation needs no second copy and costs only metadata. Failure is fully isolated: a bad run never produces a partially visible table, and consumers never observe intermediate states. Because the publish is an ordinary catalog commit, every engine sees the same atomic transition, and the audit plane records it as a single event.

10. Access Control and Policy Enforcement

10.1 Model

Access control has two levels. **Coarse** access (can this principal read this namespace or table at all?) is enforced by Polaris RBAC and by credential vending: a principal without a catalog grant cannot obtain S3 credentials for the table. **Fine** access (which rows and columns?) is enforced by OPA policies applied in the query engine: row filters

for DSA-limited rows in shared reference tables, and column masks for quasi-identifiers in human-subject data (e.g. date of birth generalised to year for Analyst roles).

10.2 Enforcement points

Engine / surface	Coarse enforcement	Fine enforcement
Trino	Polaris grants on table load; vended credentials	Trino OPA access-control plugin: row filters and column masks from the RDLR Rego bundle
Spark (EMR on EKS)	Polaris grants on table load; table-scoped vended credentials	RDLR Spark extension evaluates the same OPA bundle and injects row filters and column masks during query planning
Scholar Assist	Entitlement Resolver pre-filter on <code>project_id</code> / <code>dataset_id</code> (Section 14.3)	Not applicable — tabular rows are never embedded
Direct S3	Denied by bucket policy except vended credentials	Not applicable

10.3 Policy bundle

The Rego bundle is generated from the Project Registry on every change and at least every five minutes, signed, versioned, and served to Trino coordinators and Spark drivers. Every decision is logged with the bundle version, so any historical access can be re-evaluated against the policy in force at the time.

10.4 Embargo enforcement in the data plane

An embargoed dataset is readable only by roles on its owning project; DSA-based grants to embargoed datasets are suspended until lift. A daily job at 00:00 UTC lifts embargoes whose date has passed by clearing the embargo flag in the Registry, which regenerates grants and the bundle.

11. Analysis Environments

JupyterHub runs on the shared EKS cluster with one pod per user server, user home directories on Amazon EFS (one access point per `wid`), and project scratch on S3 under `_scratch/<wid>/`. Users choose from curated images (Python, R, Julia) that include the Trino client, PySpark configured for EMR on EKS, and the Iceberg REST client.

Credentials. Each notebook server receives the user's own OIDC tokens from JupyterHub's authentication state; a sidecar refreshes the access token before expiry into a tmpfs file readable only by the user's kernel, so long-running kernels and multi-day Spark sessions keep working without re-login. Trino is called with the user's own bearer token, so Trino and OPA evaluate the user's `wid` and project roles. Spark sessions obtain table-scoped credentials from Polaris for the user's identity, with session tags carrying `wid` and `project_id`, so CloudTrail data events attribute every object read to the individual. No shared or faculty-level credentials exist in notebook pods. The user's refresh token is revoked when their last project role is removed or their SSO account is disabled.

Resources. Default profile 4 vCPU / 32 GB; GPU and large-memory profiles on request, billed to the selected project. Idle servers are culled after 8 hours; Spark sessions launched from notebooks run on EMR on EKS with the project tag set from the user's selected project.

12. Embargo and Data-Sharing Agreements

12.1 Embargo

An embargo record holds: subject (dataset or publication), reason (PUBLICATION, PATENT, SPONSOR), lift date, set-by and justification. The PI may set an embargo at registration or at any time before publication, and may extend it up to twice (for example, while a patent filing is pending). Sponsor embargoes are set by the Research Contracts Office and can be changed only by that office.

Lineage is captured with OpenLineage from Spark and Trino. When a publish commit creates or updates a dataset whose inputs include an embargoed dataset, the Registry applies the embargo to the derived dataset in accordance with FR-6, and the derived dataset is treated identically to its source for access and discovery purposes.

12.2 Data-sharing agreements

DSAs are executed by the Research Contracts Office. RDLR does not draft or negotiate them; it enforces their structured terms.

```
DRAFT  ->  ACTIVE  ->  EXPIRED
          |
          +-->  TERMINATED  (early termination by either party)
```

Registration (DRAFT):

- Contracts Office uploads the executed agreement and records structured terms: partner, direction (inbound / outbound), datasets covered, permitted users (wid), permitted purposes, term start / end, onward-sharing prohibition, publication-review clause.

Activation (ACTIVE):

- Research Data Steward verifies the structured terms against the agreement.
- Inbound data ingested through a dedicated source; datasets tagged with `dsa_id` and `tier >= Restricted`.
- Permitted users granted the Analyst role on the covered datasets.

Expiry (EXPIRED) / Termination (TERMINATED):

- DSA record status updated; covered datasets excluded from new access requests.
- Contracts Office notified to confirm closure with the partner.

Outbound sharing. Outbound DSAs use the Data Release workflow: the PI requests a release of named table snapshots; a Research Data Steward approves; the platform exports the snapshot to a partner-specific, time-limited S3 access point with object-level logging. Releases are recorded against the DSA and appear in the audit plane.

13. Consent Withdrawal

When a human-subject participant withdraws consent, the PI registers the withdrawal in the Project Registry. The workflow is:

1. Register number.	PI records: <code>project_id</code> , participant pseudonym, effective date, IRB protocol number.
2. Resolve	Registry identifies affected tables: every table in the project whose data contract declares a <code>participant_key</code> column, plus tables derived from them (via lineage, Section 12.1).
3. Erase	Erasure job (Spark, EMR on EKS) per table: <pre>DELETE FROM <table> WHERE <participant_key> = :pseudonym</pre> Human-subject tables are configured with <code>write.delete.mode = copy-on-write</code>, so affected data files are rewritten in the erasure commit itself rather than masked by delete files. File collections: manifest rows carrying the pseudonym are deleted and the corresponding S3 objects are deleted.
4. Purge history	a. Publication tags: for each pub-tag whose snapshot contains the participant, a branch is created from the tagged snapshot, the same <code>DELETE</code> is applied on that branch, and the tag is re-pointed to the branch head. The Registry keeps a provenance record (DOI, original snapshot id, withdrawal id). b. <code>expire_snapshots</code> is called with the explicit ids of every snapshot that references a rewritten data file; <code>remove_orphan_files</code> then runs with a zero-day window scoped to the affected dataset prefixes. c. Every version (current and noncurrent) of each removed data file and collection object is deleted by version id, in us-east-1 and in the us-west-2 replica bucket.
5. Verify	For each affected table: zero rows for the pseudonym on the current snapshot and on time travel to every remaining snapshot and tag. <code>ListObjectVersions</code> on every removed key returns nothing in either region.
6. Close	Withdrawal record set to <code>CLOSED</code> ; audit event with table list, snapshot ids expired, row counts and object-version counts.
Target: steps 2-6 complete within 21 days of registration.	

Re-running a published analysis after a withdrawal reproduces the original analysis minus the withdrawn participant; the provenance record states this explicitly, which is the behaviour the approved IRB protocols require. The erasure job runs under a dedicated role that may expire snapshots and delete object versions only under human-subject dataset prefixes.

14. Scholar Assist — Retrieval-Augmented Generation

14.1 Corpus and chunking

Scholar Assist indexes text-bearing sources from Public, Internal and Restricted tiers: publications and theses (full text from CRIS and the Institutional Repository), ELN entries, dataset documentation (READMEs, codebooks, data

dictionaries), data management plans and protocols, and documentation from code repositories. Controlled-tier content is never indexed; tabular rows from any tier are never embedded.

Chunking is structure-aware (headings, ELN entry boundaries, code-doc blocks), targeting 400–600 tokens with a 64-token overlap. Each chunk record holds:

chunk_id	stable hash of (source_id, source_version, offsets)
source_id	publication, ELN entry, dataset or document identifier
source_version	Iceberg snapshot id (for manifest-backed sources) or S3 object version id
project_id	owning project ("public" pseudo-project for Public-tier sources)
dataset_id	dataset the source documents, if any (null for ELN entries and publications)
source_type	PUBLICATION ELN DATASET_DOC PROTOCOL CODE_DOC
section_path	heading path within the source
char_start/end	offsets into the source text
text_sha256	hash of chunk text
vector	1,024-dim dense embedding

No classification or embargo label is copied onto the chunk; entitlement is resolved at query time from `project_id` and `dataset_id` (Section 14.3), so changes to roles or embargoes take effect without re-indexing.

14.2 Embedding

Embeddings are produced by BAAI bge-m3 (1,024-dimensional dense vectors), self-hosted on an indexing GPU node group of 8 × g5.2xlarge inside the data account. Measured throughput is about 250 chunks per second per node at 512-token chunks. Query embedding and reranking run on a separate online node group of 2 × g5.2xlarge so that bulk indexing never competes with interactive queries.

14.3 Index and retrieval

The index is an Amazon OpenSearch Service domain with a k-NN field (faiss HNSW, M = 16, ef_construction = 128) and a BM25 text field over the same chunks. Retrieval for a query:

1. Entitlement	Resolver computes the user's entitled set: P = projects where the user is PI or Member (all source types) + "public" D = datasets granted to the user as Analyst or Viewer, or via an active DSA, excluding embargoed datasets (DATASET_DOC only) Cached per user for 60 s.
2. Search	Filtered k-NN (k = 100, ef_search = 256) and BM25 (k = 100), both with the filter <code>project_id IN P OR (dataset_id IN D AND source_type = DATASET_DOC)</code> , executed as efficient (pre-)filtering. Results fused by reciprocal rank fusion.
3. Rerank	Top 50 by RRF reranked by a cross-encoder (bge-reranker-v2-m3).
4. Assemble	Top 8 chunks, with source metadata, passed to generation.

Because the filter is applied inside the search, a non-entitled chunk is never scored, never returned and never logged as retrieved for that user (FR-9, P2).

14.4 Generation and citations

Generation uses a commercial frontier LLM through the vendor's enterprise API (public endpoint, zero-data-

retention terms, no training on customer inputs). The prompt contains the user question and the top 8 reranked chunks with their citation keys. The same model serves all eligible tiers (Public, Internal and Restricted); Controlled content is never indexed and so never reaches the model.

Every answer carries citations of the form [n] that map to `chunk_id`, and from there to `source_id` and `source_version`. A citation link opens the source at exactly that version in the portal; entitlement is re-checked when the link is opened.

14.5 Answer cache

Scholar Assist caches generated answers, with their citations, in Amazon ElastiCache (Redis), keyed by faculty and query embedding. An incoming query whose embedding has cosine similarity ≥ 0.97 with a cached query from a user in the same faculty is served the cached answer and citations directly, skipping retrieval and generation. Entries have a 24-hour TTL, and the cache for a faculty is flushed when any dataset in that faculty changes classification tier. In the pilot the hit rate was 31%, concentrated in the first weeks of term when members of the same lab groups ask similar questions; this reduces LLM spend by roughly a third.

14.6 Latency budget

NFR-3 is met on cache misses by the following per-stage budget. Pilot figures were measured on a 20M-chunk index with the production instance types.

Stage	p95 budget	Basis
Authentication and entitlement resolution	60 ms	Entitled set cached per user for 60 s; a miss costs one Registry read and one Polaris namespace listing (pilot p95 48 ms on miss).
Query embedding	30 ms	bge-m3, single query ≤ 64 tokens on A10G; pilot p95 22 ms.
Hybrid retrieval (filtered k-NN + BM25, RRF)	150 ms	Pilot p95 95 ms; budget leaves 50% headroom for the larger production index. Re-measured at full scale in Build Phase 6.
Rerank (50 pairs)	120 ms	bge-reranker-v2-m3, batched 50×512 tokens on A10G; pilot p95 104 ms.
Retrieval subtotal	360 ms	Within the 400 ms NFR-3 retrieval budget.
Prompt assembly	20 ms	Template fill and citation map.
LLM time-to-first-token	1,600 ms	Vendor p95 TTFT for prompts of 5–6k tokens, measured over four pilot weeks (1.1–1.5 s).
Total to first token	≈ 1.98 s	Within 2.5 s, with about 0.5 s headroom.

Summing per-stage p95 values over-estimates the end-to-end p95, so the budget is conservative. Answers are streamed; full-answer completion time is not bounded by NFR-3 and is reported separately.

14.7 Model lifecycle and re-embedding

Embedding model upgrades (for example, a new bge-m3 release) and changes to chunking parameters both require re-embedding. When either changes, the Indexer re-embeds the full corpus in place: each chunk's vector is overwritten in the live index with the new model's output, in source-modified order (most recent first), so the newest content benefits first. Query embedding switches to the new model at the start of the run. At about 2,000 chunks per second across the indexing node group, a full pass over 180 million chunks completes in about 25 hours; incremental indexing of newly published sources is paused for the duration so that the run has the whole node group.

15. Vector Index Sizing

Corpus estimate

Source	Items	Avg. chunks per item	Chunks
Publications and theses (full text)	610,000	44	26.8M
ELN entries	11.8M	8	94.4M
Dataset documentation (READMEs, codebooks, data dictionaries)	1.3M	14	18.2M
DMPs, protocols, grant narratives	210,000	19	4.0M
Code repository documentation	2.4M files	15	36.0M
Total			≈ 180M

OpenSearch capacity

Native memory for a faiss HNSW index is estimated with the OpenSearch formula $1.1 \times (\text{bytes per vector} + 8 \times M)$ per vector. Full-precision float32 vectors would need $1.1 \times (4 \times 1,024 + 8 \times 16) \approx 4,650$ bytes per vector, or ≈ 836 GB per copy. The index therefore uses the faiss SQfp16 scalar-quantisation encoder, which stores each dimension in 2 bytes.

Metric	Value
Per-vector memory (SQfp16, M = 16)	$1.1 \times (2 \times 1,024 + 8 \times 16) \approx 2,394$ bytes
Per copy (180M vectors)	≈ 431 GB
With one replica	≈ 862 GB

Metric	Value
Usable k-NN memory per r7g.8xlarge.search	(256 GB – 32 GB JVM heap) × 50% circuit-breaker limit ≈ 112 GB
Data nodes required at launch	862 / 112 ≈ 7.7 → 8; deployed with 10 for headroom
Growth	≈ 2.1 TB with replica by year 3 (35% per year) → ≈ 19 data nodes; scaled annually
Dedicated masters	3 × m7g.large.search
Shards	20 primaries, 1 replica each

SQfp16 retrieval quality is checked against a float32 baseline on a 5M-chunk sample in Build Phase 6.

Queries are always filtered by the entitled project set; for a typical researcher this is 3–6 projects plus the public pseudo-project and a handful of granted datasets, so the effective candidate set is a small fraction of the index.

16. Audit Plane

The audit plane implements FR-14 and NFR-8. It sits in a dedicated AWS account (`rdlr-audit`) administered by Information Security, not by RCDS.

Sources

Source	Content
CloudTrail S3 data events	Every GetObject / PutObject / DeleteObject on data buckets, with the vended-credential session tags (principal wid, table, purpose).
Trino event listener	Query id, principal, SQL text, tables and columns read, OPA bundle version, row-filter and mask policies applied, outcome.
Polaris events	Table loads, commits, grant changes, credential-vending events.
OPA decision logs	Input, decision and bundle version for every decision.
Scholar Assist retrieval log	Principal, query text, entitled project set, chunk ids retrieved, chunk ids cited, cache hit or miss.
Project Registry events	Role, embargo, DSA, tier and withdrawal changes, with actor and justification.

Pipeline and storage

Events are shipped by Amazon Data Firehose into an S3 bucket in the audit account with **S3 Object Lock in compliance mode** and a seven-year default retention. Compliance mode means no principal, including the audit account root user, can shorten retention or delete an object before it expires. Each day a manifest of that day's objects with their SHA-256 hashes is written, chained to the previous day's manifest hash, and signed with an asymmetric KMS key held in the audit account. Verification re-computes the chain from any starting day.

The audit account is queried through Athena by the Audit Office and Research Integrity Office. RCDS operators have no write, delete or policy-change permissions in it; cross-account delivery uses a Firehose role that can only PutObject.

Content boundaries

Audit records contain identifiers, never result rows or file contents. Human-subject tables are keyed by project pseudonyms, so the most that an audit record can carry about a participant is a pseudonym appearing in SQL text. The approved IRB protocols treat audit metadata as a separate record class with its own retention (NFR-7 and NFR-8 are written accordingly), so erasure of research data does not require, and must not attempt, modification of the audit store.

17. Cost Model and Chargeback

17.1 Run-rate estimate at launch scale

Line	Basis	USD / month
Primary storage (2 PB)	S3 Intelligent-Tiering, us-east-1 (see below)	8,950
Replica storage (2 PB)	S3 Glacier Instant Retrieval, us-west-2, at USD 0.004/GB-month	8,000
Replication transfer	≈ 55 TB/month new data, inter-region transfer	1,100
Trino	Two clusters, Graviton, on-demand coordinators, 60% spot workers	9,800
Spark (EMR on EKS)	Ingestion and batch, spot-heavy, incl. us-west-2 overflow cluster	7,200
JupyterHub	Node groups incl. GPU profiles at observed pilot usage	4,300
OpenSearch	Domain in Section 15 (10 data nodes, 3 masters)	23,900
GPU inference	8 × g5.2xlarge indexing (scaled to 50% duty) + 2 × g5.2xlarge online	5,800

Line	Basis	USD / month
LLM API	≈ 180,000 answers/month at pilot token profile (≈ 5.5k input, 0.7k output tokens), 31% served from cache	3,700
Catalog (two regions, Aurora), Registry, EKS control plane, networking		4,600
Audit plane	Firehose, S3 Object Lock, Athena	1,200
Total		78,550

Primary storage. S3 Intelligent-Tiering moves objects between access tiers automatically based on access patterns: Frequent Access (USD 0.023/GB-month), Archive Instant Access after 90 days without access (USD 0.004/GB-month), and Deep Archive Access after 180 days without access (USD 0.00099/GB-month), with no retrieval charges and no change in access latency for the query engines. Access telemetry from the faculty file shares shows 12% of bytes accessed within any 90-day window, 28% last accessed between 90 and 180 days ago, and 60% not accessed for more than 180 days. The blended cost is therefore $0.12 \times 2,000,000 \text{ GB} \times 0.023 + 0.28 \times 2,000,000 \text{ GB} \times 0.004 + 0.60 \times 2,000,000 \text{ GB} \times 0.00099 \approx \text{USD } 5,520 + 2,240 + 1,190 \approx \text{USD } 8,950$ per month, within NFR-9.

17.2 Attribution

Cost	Attribution key
Storage	S3 Inventory and Storage Lens by <project_id> prefix, daily
Trino	CPU-seconds per query from the event listener; project from the session property <code>rdlr.project</code> (defaults to the user's primary project)
Spark	EMR on EKS job tags (project_id)
JupyterHub	Pod labels (project_id selected at server start) × node-hour cost
OpenSearch, GPU inference, LLM API	Scholar Assist queries per project (entitled-set primary project of the user), weighted by tokens
Shared services (catalog, Registry, EKS control plane, networking, audit)	Allocated pro rata to projects by their direct spend in the month

Unallocatable residue (idle capacity that cannot be traced to any project) is reported on an RDLR-shared line, funded by the central research computing budget.

17.3 Rate card and statements

A rate card (per TB-month by tier, per Trino CPU-hour, per Spark vCPU-hour, per notebook profile-hour, per 1,000

Scholar Assist answers) is published annually. Monthly statements are generated from the AWS Cost and Usage Report joined to the attribution keys, delivered to PIs and to the Office of Sponsored Programs, and posted to grant accounts where the sponsor permits direct charging.

18. Resilience and Disaster Recovery

Within region

- EKS spans three Availability Zones; Polaris, the Entitlement Resolver and Scholar Assist API run multi-replica with pod anti-affinity.
- Trino runs two clusters (interactive and batch) behind Trino Gateway; the gateway fails interactive traffic over to the batch cluster if the interactive coordinator is lost.
- OpenSearch runs across three AZs with dedicated masters.
- The JupyterHub hub pod is a single replica; a hub restart does not terminate running user servers, which reconnect when the hub returns.
- The Project Registry runs on Multi-AZ RDS.
- Table-level recovery (NFR-11, 4 h) uses Iceberg snapshot rollback, a metadata-only operation completing in seconds.

Region loss

- All data buckets replicate to us-west-2 with S3 Cross-Region Replication into S3 Glacier Instant Retrieval (Section 17.1). Iceberg metadata files live under the same prefixes and replicate with the data.
- Iceberg `s3FileIO` maps each bucket to an S3 Multi-Region Access Point (`s3.access-points.<bucket>`), so table metadata paths remain valid after failover; MRAP failover controls switch routing to the us-west-2 replicas.
- The us-west-2 Polaris deployment is already live (Section 8.1); catalog failover is a Route 53 change. Tables resolve to the latest metadata pointer replicated by DMS, and data and metadata files arrive through CRR.
- The audit account replicates its Object Lock bucket to us-west-2 with Object Lock retained on the replica.
- RTO target 24 h, exercised twice yearly.

19. Prior Art and Reference Architecture

There is no off-the-shelf platform that combines a lakehouse, project-scoped research governance (embargo, DSA, consent withdrawal) and entitlement-aware RAG. The closest analogues each cover part of the problem.

Reference	What it governs	Concept match	Gap
Databricks Unity Catalog	Tables, files and ML assets in a lakehouse	Catalog RBAC, row filters and column masks, lineage, audit	Catalog/workspace-centric rather than project-centric; no embargo or DSA semantics; managed service tied to one compute platform.
AWS Lake Formation	Glue Data Catalog	Row and cell-level security,	Strongest with AWS-native engines; no

Reference	What it governs	Concept match	Gap
	tables on S3	tag-based access control, cross-account sharing	embargo, DSA or consent model.
Apache Polaris	Iceberg REST catalog	Multi-engine catalog, RBAC, credential vending	No row/column policy; no research-governance concepts (supplied here by the Registry).
OpenMetadata	Discovery, glossary and lineage	Search UI, lineage graphs, ownership	Discovery only; not an enforcement point.
Dataverse / Figshare / institutional repositories	Published datasets with DOIs	Embargo dates, access requests, DOI minting	Publication-oriented; no analysis engines or fine-grained policy.
RAG frameworks (LlamaIndex, LangChain-style pipelines)	Retrieval and generation orchestration	Chunking, hybrid retrieval, reranking	No entitlement model; access control left to the application.

What we borrow

From	Borrowed for RDLR
Unity Catalog / Lake Formation	Separation of coarse catalog grants from fine row/column policy; policy-version stamping on audit records.
Dataverse	Embargo record shape (reason, lift date) and access-request workflow.
OpenMetadata	Discovery UI and lineage visualisation (used directly).
Iceberg community WAP practice	Branch-based write-audit-publish (Section 9).

What is genuinely new

- Project-scoped entitlements spanning SQL engines, notebooks and a RAG service from one Registry.
- Embargo and DSA semantics as first-class access-control inputs.
- Consent-withdrawal erasure as a governed workflow over Iceberg tables and file collections.
- Citations that resolve to an exact source version through Iceberg snapshots.

20. Confirmed Decisions

Decision	Answer
Cloud and primary region	AWS, us-east-1; DR region us-west-2
Table format	Apache Iceberg v2, Parquet / ZSTD
Catalog	Apache Polaris (Iceberg REST), active in us-east-1 and us-west-2; Aurora PostgreSQL metastores with bidirectional DMS replication
Fine-grained policy	OPA as the single policy decision point for Trino, Spark and Scholar Assist; Trino OPA plugin and RDLR Spark extension; Lake Formation not used
Ingestion pattern	Write-audit-publish with Iceberg branches
Snapshot retention	400 days; publication tags retained indefinitely; both subordinate to the consent-withdrawal purge (Section 13)
Classification tiers	Public / Internal / Restricted / Controlled; Controlled in separate enclave account
Notebook credentials	Per-user OIDC tokens with sidecar refresh; no shared or faculty-level service principals
Embedding model	BAAI bge-m3, 1,024-dim, self-hosted on g5 instances
Vector store	Amazon OpenSearch Service, faiss HNSW (M = 16) with SQfp16 encoding, hybrid with BM25; 10 data nodes at launch
Generation model	Commercial frontier LLM via vendor enterprise API, zero-data-retention terms, for Public, Internal and Restricted content
Answer cache	ElastiCache, faculty-scoped, cosine ≥ 0.97 , 24 h TTL
Re-embedding strategy	Full in-place re-embed on model or chunking change
Primary storage class	S3 Intelligent-Tiering
Audit store	Separate account, S3 Object Lock compliance mode, 7 years, hash-chained daily manifests
Discovery Portal	OpenMetadata; all registered datasets listed with descriptive metadata
DR	CRR to us-west-2 (Glacier Instant Retrieval), MRAP failover, active catalog in both regions
External collaborators	InCommon / eduGAIN federation; Analyst role only

Decision	Answer

21. Pending Backlog

Items confirmed as in scope but not yet fully designed:

1. **Spark policy enforcement.** Determine whether row filters and column masks can be enforced reliably inside Spark on EMR on EKS, given that users control their own Spark sessions (and can disable planner extensions) and that vended credentials grant file-level access to whole tables. If they cannot, AWS Lake Formation becomes the enforcement point for Spark, and, to keep a single policy source, Lake Formation would replace the Trino OPA approach as well. Security Architecture review scheduled for Q1.
2. **Controlled enclave notebooks.** Design of a notebook environment inside the Controlled account (currently SQL and batch Spark only).
3. **Outbound DSA egress controls.** Whether time-limited access points are sufficient or a managed transfer service is required by some partners.
4. **Multilingual retrieval evaluation.** Benchmark bge-m3 against alternatives on non-English theses (about 9% of the corpus).
5. **HPC integration.** Direct Iceberg access from Slurm jobs with per-job vended credentials.
6. **Retention of landing-zone replay logs** beyond 14 days for DR replay.
7. **Scholar Assist evaluation harness.** Golden question sets per faculty for answer faithfulness and citation precision.
8. **ORCID-based access requests** for external collaborators without federation.

22. Validation and Acceptance Criteria

Each requirement from Section 2 is validated by a specific method with a concrete pass/fail acceptance criterion. This table is the basis for the test plan in Build Phase 8.

22.1 Functional requirements

ID	Validation method	Acceptance criteria
FR-1	Source onboarding test	Three representative sources (tabular CSV, imaging collection, ELN export) ingest end-to-end through registered pipelines; an unregistered pipeline's write to a data bucket is denied.
FR-2	Registration gate test	A publish for a dataset without project, PI or tier is rejected by the catalog commit hook.
FR-3	WAP isolation test	During validation, a reader on main sees the pre-run snapshot; a run failing each validation check (a–e) leaves main unchanged.

ID	Validation method	Acceptance criteria
FR-4	Entitlement matrix test	For each role and for a non-member in the same faculty, read attempts via Trino, Spark and direct S3 return the expected allow/deny outcome.
FR-5	Embargo confidentiality test	A user outside the project searching the Discovery Portal, the catalog APIs and Scholar Assist for the exact title of an embargoed test dataset receives no result.
FR-6	Derived-embargo test	A Spark job deriving a table from an embargoed source produces a derived dataset that is embargoed.
FR-7	DSA gating test	A user not named on an active DSA is denied access to the covered dataset even when holding a project role; a named user is allowed.
FR-8	Citation resolution test	For 100 answers, every citation resolves to a chunk, source and source version, and opening the link shows the cited text at that version.
FR-9	Retrieval entitlement test	User A (project member) and user B (non-member) issue the same 100 queries against a seeded corpus; the retrieval log shows no chunk from A's project among B's retrieved chunk ids.
FR-10	Access-request flow test	A request from the Discovery Portal creates a Registry ticket; PI approval creates the role assignment within 5 minutes.
FR-11	Reproducibility test	A query against a pub- tag created 400+ days earlier (simulated clock) returns the original result checksum.
FR-12	Withdrawal workflow test	A withdrawal registered for a seeded participant completes steps 2–5 and is CLOSED within 7 days.
FR-13	Statement generation test	A statement is generated for every active project, mapped to its cost centre or grant account.
FR-14	Audit completeness test	For a scripted sequence of reads, writes, grant changes, an embargo change, a DSA activation and a withdrawal, the audit plane contains exactly one event per operation with all required fields.
FR-15	Notebook access test	From a notebook, a user can query an entitled dataset via Trino and Spark and is denied a non-entitled dataset.

22.2 Non-functional requirements

ID	Validation method	Acceptance criteria
NFR-1	Scale test	Synthetic catalog of 2 PB metadata equivalent and 1.1B objects; 600 concurrent sessions sustain the interactive SLO for 2 h without errors.

ID	Validation method	Acceptance criteria
NFR-2	Availability measurement	Monthly availability from synthetic probes (SQL, notebook spawn, Scholar Assist query) every minute; $\geq 99.9\%$ over three consecutive months in pilot.
NFR-3	Latency benchmark	At 20 QPS on the full production index, p95 TTFT ≤ 2.5 s and p95 retrieval ≤ 400 ms on cache misses.
NFR-4	Freshness test	Instrument micro-batch published ≤ 60 min after landing; a new ELN entry retrievable ≤ 6 h after publication.
NFR-5	Egress inspection	VPC flow logs and egress proxy logs over a two-week pilot show no Restricted or Controlled payloads to non-Westmoor endpoints.
NFR-6	Key configuration check	AWS Config rule confirms every CUI-tagged bucket uses a CMK with a 90-day rotation period.
NFR-7	Erasure test	For a seeded participant, 30 days after registration: zero rows for the pseudonym in every affected table on the current snapshot and on time-travel queries to every retained snapshot and tag; zero objects (current or noncurrent) for the participant's files.
NFR-8	Audit integrity test	An attempt to delete or overwrite an audit object fails for every role including root; the daily hash chain verifies end-to-end for 30 days.
NFR-9	Cost model check	Projected monthly cost from the provisioned configuration and pilot usage is $\leq$ USD 10,000 primary storage and $\leq$ USD 80,000 total.
NFR-10	Chargeback attribution check	For the month under test: (a) the report total reconciles to within 1% of the AWS Cost and Usage Report total for the platform accounts; (b) the RDLR-shared line is $\leq 5\%$ of that total; (c) for a random sample of 20 projects, every statement line traces to tagged resources or attribution-key records belonging to that project.
NFR-11	DR exercise	Region-failover exercise restores catalog and data access in us-west-2 within 24 h with data loss ≤ 24 h; a corrupted table is rolled back within 4 h.
NFR-12	Onboarding dry run	A new faculty source is onboarded with one source YAML and one data contract and no code change.

23. Readiness Assessment

This section answers a direct question: could an engineering team, or an AI coding assistant, build each component from this document alone? The honest answer is partial.

Component	Ready?	What would still need to be decided
S3 layout, buckets, KMS keys	Ready	Nothing further; layout and policies are specified (Sections 4, 7).
Iceberg conventions and table maintenance	Ready	Settings and jobs specified (Section 7.2).
WAP ingestion framework	Ready	Steps, checks and failure behaviour specified (Section 9). Data contract YAML schema needs writing out but follows directly.
Polaris catalog and RBAC mapping	Ready	Deployment and role mapping specified (Section 8).
Project Registry	Mostly ready	Entities and states specified; REST API contract not yet written.
OPA policy bundle and Trino plugin	Ready	Enforcement points and bundle lifecycle specified (Section 10).
RDLR Spark extension	Mostly ready	Interface defined; Rego-to-Catalyst translation to be implemented.
JupyterHub environment	Ready	Images, credentials and profiles specified (Section 11).
Embargo and DSA lifecycle	Mostly ready	States specified; Registry UI for Contracts Office not designed.
Consent withdrawal	Ready	Workflow specified (Section 13).
Scholar Assist retrieval	Ready	Pipeline, filters and budget specified (Section 14).
Scholar Assist evaluation	Not ready	Golden sets and metrics in Pending Backlog item 7.
Audit plane	Ready	Sources, storage and integrity mechanism specified (Section 16).
Chargeback	Mostly ready	Attribution keys specified; statement format to be agreed with Office of Sponsored Programs.
DR	Mostly ready	Mechanisms specified; runbooks to be written.
Controlled enclave notebooks	Not ready	Pending Backlog item 2.

Overall. The storage, catalog, ingestion, policy and audit layers can be built directly. Before Scholar Assist goes beyond pilot, the evaluation harness (backlog item 7) must exist so that retrieval quality changes can be measured.

24. Build Phases

#	Phase	Depends on open items?
1	Foundations — AWS accounts (data, audit, Controlled enclave), buckets, KMS, IdP integration, Project Registry	No
2	Audit plane — Firehose, Object Lock bucket, manifests, Athena views (built before any data lands)	No
3	Catalog and storage — Polaris, Iceberg conventions, table maintenance jobs	No
4	Ingestion — WAP framework, classification scanner, first three sources	No
5	Access control and analysis — OPA bundle, Trino plugin, Spark extension, JupyterHub	No
6	Scholar Assist — indexer, embedding fleet, OpenSearch, retrieval, generation, citations, cache	Partially — evaluation harness (backlog 7)
7	Governance workflows — embargo, DSA lifecycle, consent withdrawal, Discovery Portal, chargeback	No
8	Validation and pilot — Section 22 test plan; two pilot faculties (Life Sciences, Engineering) for one term	Follows Section 22
9	Wave rollout — remaining nine faculties in three waves; file-share decommissioning	Pilot exit criteria met

This document, together with the RDLR Conceptual Design, represents the design phase in full. No implementation has begun.